

CURSO DE PYTHON

**Corporación Ecuatoriana para el Desarrollo de la
Investigación y la Academia**

Ing. María Fernanda Granda J., PhD

22-07-2026

**“Conectando ideas,
transformando
sociedades”**

Tema 4: Programación Orientada a Objetos (POO) en Python (II parte)

- Encapsulamiento
- Herencia
- Polimorfismo
- Actividades autónomas

Propiedades

Actividad inicial

- Para manejar los atributos de un objeto, podemos utilizar @property para permitir a un método ser accedido como un atributo, omitiendo así el uso de paréntesis vacíos.
- Los paréntesis son vacíos cuando un método no tiene parámetros.

Sin property

```
def area(self):  
    return 3.1416 * self.radio ** 2
```

```
print(f'El área del cilindro es {cilindro1.area()}')
```

Usando property

```
@property  
def area(self):  
    return 3.1416 * self.radio ** 2
```

```
print(f'El área del cilindro es {cilindro1.area}')
```

Propiedades

- Para modificar una propiedad, se necesita un método **.setter()**

`@Nombre_Propiedad.setter`

```
@property
def area(self):
    return pi * self.radio ** 2

#Modificando la propiedad
@area.setter
def area(self, area):
    self.radio = (area / pi) ** 0.5
```

Herencia

- Permite que una clase (subclase) herede atributos y métodos de otra clase (superclase), esto permite la reutilización del código.
- Permite **compartir** un **comportamiento** que es **común** en la **jerarquía**
- Al padre se le conoce como **superclase** y al hijo como **subclase**
- La herencia permite crear entonces nuevas clases a partir de otras
- **Extiende** el **comportamiento** de clases existentes, dando mayor **flexibilidad** y **modularidad** en el diseño del Sistema.

Herencia

```
class Poligono:
    def __init__(self, numero_lados):
        self.numero_lados = numero_lados
        self.lados = [0] * numero_lados # propiedad de duplicación para inicializar con 0 la lista de los lados

    def ingresar_lados(self):
        self.lados = [float(input(f'Lado {i+1}: ')) for i in range(self.numero_lados)] #list comprehension

    def mostrar_lados(self):
        for i in range(self.numero_lados):
            print(f'El lado {i+1} es {self.lados[i]}')
```

```
from poligono import Poligono

class Triangulo(Poligono):
    def __init__(self, color):
        super().__init__(3)
        self.color = color

    @property
    def area(self):
        lado1, lado2, lado3 = self.lados
        s = (lado1 + lado2 + lado3) / 2
        area = (s * (s - lado1) * (s - lado2) * (s - lado3)) ** 0.05
        print(f'El área del triángulo es %.02f' % area)
```

- **super()** nos permite tener una referencia de la superclase Poligono, por lo tanto puedo llamar sus métodos sin necesidad de repetir código.

Herencia

```
from triangulo import Triangulo

if __name__ == '__main__':
    triangulo = Triangulo('azul')
    print(triangulo.lados)
    triangulo.ingresar_lados()
    print(triangulo.lados)
    triangulo.area
```

- En otro componente **principal.py**
- La salida sería algo así:

```
[0, 0, 0]
Lado 1: 3
Lado 2: 4
Lado 3: 5
[3.0, 4.0, 5.0]
El área del triángulo es 1.20
```

Actividad 1

Problema

Tenemos:

Estudiante
Profesor
Administrador

Todos tienen:

- nombre
- correo
- teléfono

Actividad 1

Solución:

Clase base.

Clase Padre

```
class Persona:  
  
    def __init__(self, nombre):  
  
        self.nombre = nombre
```

Clase Hija:

```
class Estudiante(Persona):  
  
    pass
```

Heredar atributos:

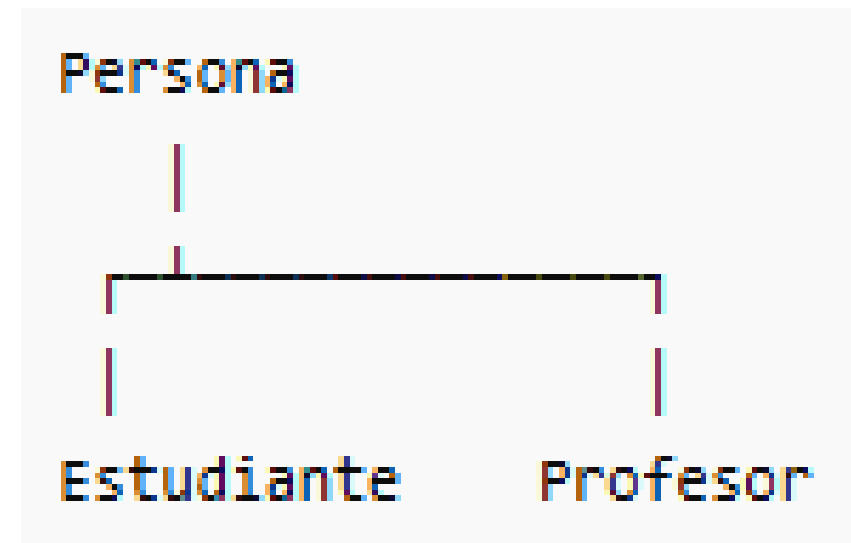
```
est = Estudiante("Pedro")  
  
print(est.nombre)
```

Actividad 1

Agregar nuevos atributos

```
class Estudiante(Persona):  
  
    def __init__(self, nombre, carrera):  
  
        super().__init__(nombre)  
  
        self.carrera = carrera
```

Diagrama UML simple



Actividad 2

Crear sistema:

Clase base

Vehiculo

Atributos:

- marca
- modelo

Clases derivadas

Automovil
Motocicleta
Camion

Agregar atributos propios

Herencia Múltiple

- Python permite herencia de múltiples superclases, dándole la posibilidad de acceder a una multitud de métodos.
- Cuando hay métodos comunes dentro de las superclases, Python le dará prioridad a las superclases más a la izquierda en la declaración

```
class A:
    def __init__(self): #1
        print('Soy la clase A')
    def a(self): #2
        print('Método heredado de clase A')
class B:
    def __init__(self): #1
        print('Soy la clase B')
    def b(self): #2
        print('Método heredado de clase B')
class C(A, B):
    def c(self): #2
        print('Método de clase C')
```

Herencia Múltiple

```
class A:
    def __init__(self):
        print('Soy la clase A')

    def a(self):
        print('Método heredado de la clase A')

class B:
    def __init__(self):
        print('Soy la clase B')

    def b(self):
        print('Método heredado de la clase B')

class C(A, B):
    def c(self):
        print('Método propio de la clase C')

a = A()
b = B()
c = C()
c.a()
c.b()
c.c()
```

- Salida

```
Soy la clase A
Soy la clase B
Soy la clase A
Método heredado de la clase A
Método heredado de la clase B
Método propio de la clase C
```

Actividad 3

Clase Empleado:

- Construye dos superclases, donde una será la clase **Empleados** que pida en consola el nombre del empleado en un método de instancia.
- La segunda clase será **Salario** que pida en consola el salario del empleado en un método de instancia
- Cree una clase hija **Designación** que herede de las dos clases anteriores y que tenga un método de instancia que designe el cargo del empleado y que muestre todos los datos del empleado (nombre, salario y designación/cargo)
- Verifique el código instanciando un objeto de la clase hija, verificando si el objeto tiene el método nombre y salario.

Encapsulamiento

Concepto: Ocultar información sensible

- Los atributos que empiezan con un guion bajo se consideran protegidos (no pueden ser accedidos directamente sino a través de un método)
- Los que comienzan con doble guion bajo son privados, es decir evita que sea sobrescrito accidentalmente por las subclases en la herencia

Atributos protegidos

```
class Cuenta:  
  
    def __init__(self):  
  
        self._saldo = 0
```

Atributos privados

```
class Cuenta:  
  
    def __init__(self):  
  
        self.__saldo = 0
```

Encapsulamiento

- Los detalles internos de un objeto deben estar ocultos fuera de su definición
- Deben ser accesibles a través de una interfaz claramente definida
- Promueve la seguridad e integridad de los datos
- Facilita el mantenimiento del código

Encapsulamiento

Uso

```
cuenta = Cuenta('123', 1000)
print(f'Clave: {cuenta.clave} - Cantidad: {cuenta.cantidad} ')
cuenta.establecer_clave('4567','123')
cuenta.establecer_cantidad(2000,'4567')
print(f'Clave: {cuenta.clave} - Cantidad: {cuenta.cantidad} ')
```

Resultado

```
Clave: 123 - Cantidad: 1000
La clave fue cambiada con éxito
La cantidad fue cambiada con éxito
Clave: 4567 - Cantidad: 2000
```

```
class Cuenta:
    def __init__(self, clave, cantidad):
        self._clave = clave
        self._cantidad = cantidad

    #GETTERS
    @property
    def clave(self):
        return self._clave

    @property
    def cantidad(self):
        return self._cantidad

    def verificar_clave(self, clave):
        return self._clave == clave

    #SETTERS
    def establecer_clave(self, clave_nueva, clave):
        if self.verificar_clave(clave):
            if len(clave_nueva)<4:
                print("La clave debe tener al menos 4 caracteres")
            else:
                print("La clave fue cambiada con éxito")
                self._clave = clave_nueva
        else:
            print("La clave ingresada no es la correcta")

    def establecer_cantidad(self, cantidad_nueva, clave):
        if self.verificar_clave(clave):
            if cantidad_nueva<=0:
                print("No se puede cambiar por una cantidad negativa")
            else:
                print("La cantidad fue cambiada con éxito")
                self._cantidad = cantidad_nueva
        else:
```

Actividad 4

Uso de métodos de acceso

```
class Cuenta:

    def __init__(self):

        self.__saldo = 0

    def obtener_saldo(self):

        return self.__saldo
```

Uso de Property

```
class Producto:

    def __init__(self):

        self.__precio = 0

    @property
    def precio(self):
        return self.__precio

    @precio.setter
    def precio(self, valor):

        if valor > 0:
            self.__precio = valor
```

Polimorfismo

Concepto: Un mismo método produce comportamientos distintos.

- Capacidad de objetos de diferentes clases de responder al mismo mensaje de manera diferente
- Permite escribir código más genérico y reutilizable

Ejemplo:

```
class Perro:  
  
    def hablar(self):  
        print("Guau")
```

```
class Gato:  
  
    def hablar(self):  
        print("Miau")
```

Polimorfismo

Uso

```
animales = [  
    Perro(),  
    Gato()  
]  
  
for animal in animales:  
    animal.hablar()
```

Resultado

```
Guau  
Miau
```

Polimorfismo

```
class Instrumento():
    def __init__(self, nombre):
        self.nombre = nombre

    def tocar(self):
        print("\033[37m El instrumento genérico Hace un sonido melodioso")

class Guitarra(Instrumento):
    def __init__(self):
        super().__init__("Guitarra")
    def tocar(self):
        print(f"\033[91m {self.nombre} esta haciendo el mejor solo usando bendings y ligados")

class Bateria(Instrumento):
    def __init__(self):
        super().__init__("Bateria")

    def tocar(self):
        print(f" \033[92m{self.nombre} está llevando un gran ritmo 4/4")

class Piano(Instrumento):
    def __init__(self):
        super().__init__("Piano")

    def tocar(self):
        print(f"\033[93m {self.nombre} nos está deleitando con una gran Blues Clásico")
```

Uso

```
instrumento = Instrumento("Básico")
guitarra = Guitarra()
bateria = Bateria()
piano = Piano()

instrumento.tocar()
guitarra.tocar()
bateria.tocar()
piano.tocar()
print("\033[37m")
```

Resultado

```
El instrumento genérico Hace un sonido melodioso
Guitarra esta haciendo el mejor solo usando bendings y ligados
Bateria está llevando un gran ritmo 4/4
Piano nos está deleitando con una gran Blues Clásico
```

Actividad 5

```
EmpleadoTiempoCompleto  
EmpleadoPorHoras  
Consultor
```

- Todos tienen

```
calcular_pago()
```

- pero cada uno lo implementa diferente

Actividad 6

Crear :

```
Figura
```

y las clases:

```
Circulo  
Rectangulo  
Triangulo
```

Cada una debe implementar:

```
calcular_area()
```

Clases Abstractas

Concepto: Una clase abstracta en Python funciona como una plantilla o "contrato" para otras clases.

- Define qué métodos deben tener las clases hijas, pero no proporciona el código real de esos métodos.
- No se pueden crear objetos directamente a partir de ella, obligando a que las subclases implementen los métodos abstractos.
- Para crear una clase abstracta se utiliza el módulo nativo abc (Abstract Base Classes), heredando de ABC y usando el decorador @abstractmethod

Ejemplo

```
from abc import ABC, abstractmethod

# Clase abstracta
class Forma(ABC):

    def __init__(self, color):
        self.color = color

    @abstractmethod
    def calcular_area(self):
        pass # No tiene implementación
```

```
# Clase hija que hereda de la clase abstracta
class Circulo(Forma):

    def __init__(self, color, radio):
        super().__init__(color)
        self.radio = radio

    # Obligatorio implementar el método abstracto
    def calcular_area(self):
        return 3.1416 * self.radio ** 2
```

```
# Uso
# forma = Forma("rojo") # Esto daría un error, no se puede instanciar
mi_circulo = Circulo("azul", 5)
print(mi_circulo.calcular_area())
```

Sistema de Gestión Académica

Objetivo

Aplicar todos los conceptos vistos

Parte 1

Clase base:

```
</> Python
```

```
Persona
```

Atributos:

```
</> Python
```

```
nombre  
correo
```

Parte 2

Herencia

```
</> Python
```

```
Estudiante  
Profesor
```

Parte 3

Encapsulamiento

```
</> Python
```

```
__promedio
```

No permitir:

```
</> Python
```

```
promedio < 0  
promedio > 10
```

Parte 4

Polimorfismo

Método:

```
</> Python
```

```
presentarse()
```

Salida:

```
</> Python
```

```
Soy estudiante de Software
```

```
</> Python
```

```
Soy profesor de Programación
```

Resultado esperado

```
</> Python
```

```
est1.presentarse()
```

```
prof1.presentarse()
```

Sistema de Biblioteca Digital

Diseñar:

```
Material
|
├─ Libro
├─ Revista
└─ Tesis
```

Implementar:

```
mostrar_informacion()
```

mediante polimorfismo

Tarea Autónoma

Proyecto: Sistema de Gestión Hospitalaria

Clase base

```
</> Python
```

```
Persona
```

Clases derivadas

```
</> Python
```

```
Paciente
```

```
Doctor
```

```
Enfermero
```

Requisitos

Herencia: Todas las clases deben heredar de Persona.

Encapsulamiento

```
</> Python
```

```
__historial_medico
```

Polimorfismo

```
</> Python
```

```
mostrar_informacion()
```

Entregable:

- Código organizado en módulos.
- Aplicación de POO.
- Diagrama UML básico.
- Evidencias de ejecución.

GRACIAS